

Supplementary Material of the article: Assessing the Impact of Public Policies on Vehicle Theft Incidence in Natal: A Spatial-Temporal Model Using Non-Homogeneous Poisson Processes

This document provides supplementary material to the article, with the objective of enabling full reproducibility of the results. Below, we describe in detail the dataset used in the study.

Installation of the STPoissonSS package

To run the model presented in the article, it is necessary to install the STPoissonSS package available on GitHub. Installation can be performed using the devtools package:

```
install.packages("devtools")
devtools::install_github("Projeto-CNPq-Clima/STPoissonSS", force = TRUE)
```

Once installed, load the package with:

```
library(STPoissonSS)
```

Data

The data used in the analysis are available in the STPoissonSS package, in the file `natal_vehicle_theft_data`. To load it in R, use the command:

```
data(natal_vehicle_theft_data)
```

The file contains three main objects:

1. `natal_theft_times`

Numeric matrix of dimension $T \times 34$, where each column represents a neighborhood of Natal and contains the event times of vehicle thefts (in seconds since “2012-01-01 00:00:00”).

- Rows correspond to individual events.
- Since the number of thefts differs across neighborhoods, each column may contain a different number of observed event times. Columns corresponding to neighborhoods with fewer thefts will have missing values (NA) in the remaining rows, while those with more thefts will be more fully populated.

2. `natal_sites_utm`

Numeric matrix of dimension 34×2 with the UTM coordinates (Easting, Northing) of the centroids of the 34 neighborhoods.

- The neighborhood order is exactly the same as in `natal_theft_times`.
- Columns: Easting and Northing.

3. `natal_covariates`

Numeric matrix of dimension 34×3 with neighborhood-level covariates.

- Each row corresponds to a neighborhood, in the same order as the other objects.
- Columns (in this order):
 - (a) `density`: population density
 - (b) `pm_stations`: number of Polícia Militar stations
 - (c) `squares`: number of public squares

Processing details

- The original event times were provided as text strings in the format “%d/%m/%Y %H:%M:%S” and converted into numeric values (seconds since “2012-01-01 00:00:00”) using the function `strptime` in R.
- To recover the original calendar datetimes, the following code can be used:

```
origin <- as.POSIXct("2012-01-01 00:00:00", tz = "America/Fortaleza")
dt <- origin + natal_theft_times[, 1] # example for the first neighborhood
```

Neighborhood order

The order of neighborhoods is consistent across all objects (`natal_theft_times`, `natal_sites_utm`, and `natal_covariates`). Names were kept in Portuguese, without translation, and follow the administrative division by zones:

- **South Zone:** Lagoa Nova; Nova Descoberta; Candelária; Capim Macio; Pitimbu; Neópolis; Ponta Negra
- **East Zone:** Santos Reis; Rocas; Ribeira; Praia do Meio; Petrópolis; Areia Preta; Cidade Alta; Mãe Luiza; Alecrim; Barro Vermelho; Tirol; Lagoa Seca
- **West Zone:** Cidade da Esperança; Quintas; Dix-Sept Rosado; Bom Pastor; Nossa Senhora de Nazaré; Felipe Camarão; Cidade Nova; Guarapes; Planalto
- **North Zone:** Igapó; Potengi; Nossa Senhora da Apresentação; Lagoa Azul; Pajuçara; Redinha

Note

For package consistency, the file and objects were named in English:

- File name: `natal_vehicle_theft_data.RData`
- Objects: `natal_theft_times`, `natal_sites_utm`, `natal_covariates`

The `bayesSTWeibull` function

The package provides the function `bayesSTWeibull`, which performs Bayesian inference for the spatiotemporal nonhomogeneous Poisson model with Weibull intensity and state-space prior. Model parameters are estimated using a Markov chain Monte Carlo (MCMC) algorithm.

Main input arguments

- **data:** Matrix or list of observed counts, following the structure of a nonhomogeneous spatiotemporal Poisson process.
- **sites:** Matrix with spatial coordinates (UTM) of neighborhoods.
- **tau_breaks:** Numeric vector with breakpoints that partition the time interval $(0, T]$.
- **Ftw, Ftm:** Arrays of covariates used to explain the latent vectors W (scale) and M (shape).

- **G, Q:** Arrays of evolution matrices for the temporal dynamics of W and M .
- **prior:** List of hyperparameters for prior distributions (means, covariances, inverse-Wishart, inverse-gamma, and gamma priors).
- **iteration:** Total number of MCMC iterations.
- **burnin:** Number of initial iterations discarded as burn-in.
- **jump:** Thinning interval, i.e., number of iterations skipped between two consecutive stored samples.

Main outputs

- Posterior samples of the latent parameters (scale, shape, intercepts, regression coefficients).
- Posterior samples of variances, covariances, and spatial dependence parameters.
- Posterior means of the W (scale) and M (shape) processes.

Example: running the model of the article

The following code illustrates how to run the proposed model using the vehicle theft data in Natal:

```
install.packages("devtools")
devtools::install_github("Projeto-CNPq-Clima/STPoissonSS", force = TRUE)
library(STPoissonSS)
data(natal_vehicle_theft_data)

natal_theft_times <- natal_vehicle_theft_data$natal_theft_times
sites <- natal_vehicle_theft_data$natal_sites_utm
covar <- natal_vehicle_theft_data$natal_covariates
covar[,4] <- (covar[,4] - mean(covar[,4])) / sd(covar[,4])
covar[,5] <- (covar[,5] - mean(covar[,5])) / sd(covar[,5])
covar[,6] <- (covar[,6] - mean(covar[,6])) / sd(covar[,6])

breaks <- c(1096, 2557, 3531)
Uns <- as.matrix(rep(1, ncol(natal_theft_times)))
X <- t(as.matrix(cbind(Uns, sites, covar)))
G <- diag(1, nrow(X))

Xm <- array(NA, dim = c(length(breaks), nrow(X), ncol(natal_theft_times)))
Zm <- array(NA, dim = c(length(breaks), nrow(X), ncol(natal_theft_times)))
Gx <- array(NA, dim = c(length(breaks), nrow(G), ncol(G)))
Gz <- Gx

for(l in 1:length(breaks)){
  Xm[l,,] <- X
  Zm[l,,] <- X
  Gx[l,,] <- G
  Gz[l,,] <- G
}

Mod30 <- bayesSTWeibull(
  data = natal_theft_times,
```

```

sites = sites,
tau_breaks = breaks,
Ftw = Xm,
Ftm = Zm,
G = Gx,
Q = Gz,
iteration = 800000,
burnin = 500000,
jump = 1
)

```

Convergence of the parameters

To evaluate the convergence of the chains generated by the MCMC algorithm, six outputs corresponding to the parameter estimates of the model were used. These outputs can be obtained in Castro Morales (2025) Castro Morales (2025b).

The first step is to load the files containing the results of the chains. Each file corresponds to an independent chain with 300k iterations:

```

## ===== Load the 6 chains (300k each) =====
load("./outputS30.RData") # Mod30
load("./outputS31.RData") # Mod31
load("./outputS32.RData") # Mod32
load("./outputS33.RData") # Mod33
load("./outputS34.RData") # Mod34
load("./outputS35.RData") # Mod35

```

The convergence of the parameters was evaluated using the diagnostics of **Gelman and Rubin (1992)**, specifically the *Potential Scale Reduction Factor (PSRF)*, as implemented in the `coda` package in R. This diagnostic compares between-chain variability with within-chain variability, providing evidence about the stabilization of the posterior distribution.

The following code was used to compute the Gelman and Rubin diagnostics for the β (**MBeta**) and Ψ (**MPsi**) parameters:

```

## ===== Convergence (coda): 6 chains, MBeta and MPsi =====
if (!requireNamespace("coda", quietly = TRUE)) install.packages("coda")
library(coda)

# ----- Helper functions -----
align_columns_list <- function(mats, what = "MBeta") {
  stopifnot(length(mats) >= 2)
  cn1 <- colnames(mats[[1]])
  if (is.null(cn1)) {
    k <- ncol(mats[[1]])
    ok <- vapply(mats, function(M) ncol(M) == k, TRUE)
    if (!all(ok)) stop(sprintf("%s: number of columns differ between chains.", what))
    return(mats)
  }
  for (i in seq_along(mats)) {
    cni <- colnames(mats[[i]])

```

```

    if (is.null(cni) || !setequal(cni, cn1))
      stop(sprintf("%s: different sets of columns between chains.", what))
    mats[[i]] <- mats[[i]][, cn1, drop = FALSE]
  }
  mats
}

equalize_lengths <- function(mats) {
  min_n <- min(vapply(mats, nrow, 1L))
  lapply(mats, function(M) M[(nrow(M) - min_n + 1):nrow(M), , drop = FALSE])
}

analyze_component_all <- function(mods, comp = c("MBeta", "MPsi"), label = NULL) {
  comp <- match.arg(comp)
  label <- if (is.null(label)) comp else label

  stopifnot(length(mods) >= 2)
  has_comp <- vapply(mods, function(m) comp %in% names(m), TRUE)
  if (!all(has_comp)) {
    cat(sprintf("[Info] '%s' missing in some chain; %s not analyzed.\n", comp, label))
    return(invisible(NULL))
  }

  mats <- lapply(mods, function(m) as.matrix(m[[comp]]))
  mats <- align_columns_list(mats, what = comp)
  mats <- equalize_lengths(mats)

  if (is.null(colnames(mats[[1]]))) {
    colnames(mats[[1]]) <- paste0(label, ".", seq_len(ncol(mats[[1]])))
    for (i in 2:length(mats)) colnames(mats[[i]]) <- colnames(mats[[1]])
  }

  mlist <- mcmc.list(lapply(mats, mcmc))
  gr <- gelman.diag(mlist, autoburnin = FALSE)

  tab <- data.frame(
    Param = colnames(mats[[1]]),
    Rhat = as.numeric(gr$psrf[, 1]),
    Rhat_UpperCI = as.numeric(gr$psrf[, 2]),
    stringsAsFactors = FALSE
  )
  mpsrf <- unname(gr$mpsrf)

  cat("\n===== \n")
  cat(sprintf("== %s: Gelman-Rubin with %d chains (all samples) == \n",
    label, length(mods)))
  cat(sprintf("Parameters: %d | Multivariate PSRF: %.4f \n \n", nrow(tab), mpsrf))
  print(tab, row.names = FALSE)

  invisible(list(table = tab, mpsrf = mpsrf, mlist = mlist))
}

# ----- Run for 6 chains -----

```

```

mods6 <- list(Mod30, Mod31, Mod32, Mod33, Mod34, Mod35)

res_beta <- analyze_component_all(mods6, "MBeta", "(MBeta)")
res_psi <- analyze_component_all(mods6, "MPsi", "(MPsi)")

```

The following are the Potential Scale Reduction Factor (PSRF) estimates for the Beta and Psi parameters

```

=====
== (MBeta): Gelman-Rubin with 6 chains (all samples) ==
Parameters: 27 | Multivariate PSRF: 1.0477

      Param      Rhat Rhat_UpperCI
(MBeta).1 1.009841    1.009890
(MBeta).2 1.047841    1.047854
(MBeta).3 1.044715    1.044731
(MBeta).4 1.160667    1.161269
(MBeta).5 1.145534    1.147825
(MBeta).6 1.174902    1.178950
(MBeta).7 1.257639    1.257677
(MBeta).8 1.078844    1.079612
(MBeta).9 1.095106    1.095721
(MBeta).10 1.179415    1.186964
(MBeta).11 1.137943    1.157061
(MBeta).12 1.164720    1.203976
(MBeta).13 1.223883    1.231663
(MBeta).14 1.192871    1.217287
(MBeta).15 1.209275    1.243841
(MBeta).16 1.129850    1.132242
(MBeta).17 1.110635    1.117585
(MBeta).18 1.109591    1.117203
(MBeta).19 1.257110    1.259831
(MBeta).20 1.234474    1.239685
(MBeta).21 1.260848    1.278834
(MBeta).22 1.096296    1.097939
(MBeta).23 1.099886    1.102362
(MBeta).24 1.022356    1.027009
(MBeta).25 1.118547    1.120727
(MBeta).26 1.125502    1.130547
(MBeta).27 1.027286    1.033293
> res_psi <- analyze_component_all(mods6, "MPsi", "(MPsi)")

=====
== (MPsi): Gelman-Rubin with 6 chains (all samples) ==
Parameters: 27 | Multivariate PSRF: 1.3270

      Param      Rhat Rhat_UpperCI
(MPsi).1 1.000221    1.000228
(MPsi).2 1.000791    1.000808
(MPsi).3 1.000308    1.000316
(MPsi).4 1.079008    1.187316
(MPsi).5 1.082085    1.192435
(MPsi).6 1.073184    1.173857
(MPsi).7 1.065547    1.156699

```

(MPsi).8	1.063315	1.150295
(MPsi).9	1.054170	1.130126
(MPsi).10	1.150863	1.341159
(MPsi).11	1.167977	1.375591
(MPsi).12	1.190988	1.423032
(MPsi).13	1.146643	1.335055
(MPsi).14	1.165564	1.377179
(MPsi).15	1.182554	1.409801
(MPsi).16	1.056084	1.134102
(MPsi).17	1.068895	1.162224
(MPsi).18	1.059950	1.143188
(MPsi).19	1.097282	1.233024
(MPsi).20	1.108242	1.256806
(MPsi).21	1.095539	1.228141
(MPsi).22	1.026695	1.049475
(MPsi).23	1.027375	1.051987
(MPsi).24	1.025667	1.048348
(MPsi).25	1.044654	1.100063
(MPsi).26	1.044877	1.101685
(MPsi).27	1.042162	1.094805

The following code tests the convergence of the parameters σ_w^2 , σ_m^2 , ϕ_w and ϕ_m :

```
## ===== Prerequisites (same as already used) =====
if (!requireNamespace("coda", quietly = TRUE)) install.packages("coda")
library(coda)

## ----- Helper functions (same) -----
align_columns_list <- function(mats, what = "MBeta") {
  stopifnot(length(mats) >= 2)
  cn1 <- colnames(mats[[1]])
  if (is.null(cn1)) {
    k <- ncol(mats[[1]])
    ok <- vapply(mats, function(M) ncol(M) == k, TRUE)
    if (!all(ok)) stop(sprintf("%s: number of columns differ between chains.", what))
    return(mats)
  }
  for (i in seq_along(mats)) {
    cni <- colnames(mats[[i]])
    if (is.null(cni) || !setequal(cni, cn1))
      stop(sprintf("%s: different sets of columns between chains.", what))
    mats[[i]] <- mats[[i]][, cn1, drop = FALSE]
  }
  mats
}

equalize_lengths <- function(mats) {
  min_n <- min(vapply(mats, nrow, 1L))
  lapply(mats, function(M) M[(nrow(M) - min_n + 1):nrow(M), , drop = FALSE])
}

analyze_component_all <- function(mods, comp =
  c("MBeta", "MPsi", "Mbw", "Mbm", "Mvw", "Mvm"),
```

```

                                label = NULL) {
## ===== Run for the 6 chains =====
mods6 <- list(Mod30, Mod31, Mod32, Mod33, Mod34, Mod35)

## ----- Convergence (Gelman-Rubin) -----
res_b_w <- analyze_component_all(mods6, "Mbw", "(Mbw)")
res_b_m <- analyze_component_all(mods6, "Mbm", "(Mbm)")
res_v_w <- analyze_component_all(mods6, "Mvw", "(Mvw)")
res_v_m <- analyze_component_all(mods6, "Mvm", "(Mvm)")

> res_b_w <- analyze_component_all(mods6, "Mbw", "(Mbw)")

=====
== (Mbw): Gelman-Rubin with 6 chains (all samples) ==
      Param      Rhat Rhat_UpperCI
(Mbw).1 1.004264      1.009572
> res_b_m <- analyze_component_all(mods6, "Mbm", "(Mbm)")

=====
==(Mbm): Gelman-Rubin with 6 chains (all samples) ==
      Param      Rhat Rhat_UpperCI
(Mbm).1 1.002228      1.004016
> res_v_w <- analyze_component_all(mods6, "Mvw", "(Mvw)")

=====
== (Mvw): Gelman-Rubin with 6 chains (all samples) ==
      Param      Rhat Rhat_UpperCI
(Mvw).1 1.062904      1.065058
> res_v_m <- analyze_component_all(mods6, "Mvm", "(Mvm)")

=====
==(Mvm): Gelman-Rubin with 6 chains (all samples) ==
      Param      Rhat Rhat_UpperCI
(Mvm).1 1.288922      1.28901

```

Convergence Analysis

The Gelman–Rubin diagnostic results provide consistent evidence of convergence for most of the parameters evaluated.

Parameters β (MBeta):

Among the 27 estimated parameters, the majority exhibit PSRF values very close to 1, indicating adequate convergence across the chains. The multivariate PSRF (1.0477) further confirms that, overall, the chains are well mixed. Although some individual parameters show values slightly above 1.1, the general pattern is satisfactory, with clear evidence of stability for most of the β coefficients.

Parameters Ψ (MPsi):

Several coordinates show strong convergence, with Rhat values extremely close to 1 (e.g., $\Psi.1$, $\Psi.2$, and $\Psi.3$). Even though the multivariate PSRF is somewhat higher (1.3270), the overall pattern suggests that many parameters are approaching convergence. With additional iterations or more chains, the convergence for these parameters is expected to improve further.

Spatial and temporal dependence parameters (ϕ_w and ϕ_m):

These parameters display excellent convergence, with PSRF values essentially equal to 1 (1.004 and 1.002, respectively). This result is particularly important, as it ensures reliability in estimating the spatiotemporal dependence effects of the model.

Variance parameters (σ_w^2 and σ_m^2):

For σ_w^2 , the PSRF value (1.06) indicates satisfactory convergence, still within an acceptable range. For σ_m^2 , although the value is above the ideal threshold (1.28), it still shows a tendency toward stabilization, which can be further improved in subsequent analyses. Importantly, this is an isolated case among the set of parameters.

Overall, the results demonstrate that the majority of the parameters achieved satisfactory convergence, particularly the β coefficients, the first Ψ components, and the dependence parameters ϕ_w and ϕ_m , which are central to the spatiotemporal structure of the model. These findings indicate that the inferences are robust, with only a few parameters requiring closer monitoring.

Reproducibility of the Tables

To estimate the Beta and Psi parameters we use the six chains employed in the convergence analysis. Below is the code to reproduce Tables 1 and 2 of the article.

```
## ----- Pooled summaries (generic) -----
summarize_pooled <- function(mods, comp, label,
                             param_names = NULL,
                             assume_3_deltas = TRUE) {
  has_comp <- vapply(mods, function(m) comp %in% names(m), TRUE)
  if (!all(has_comp)) {
    cat(sprintf("[Info] '%s' missing in some chain; %s not summarized.\n", comp, label))
    return(invisible(NULL))
  }
  mats <- lapply(mods, function(m) as.matrix(m[[comp]]))
  mats <- align_columns_list(mats, what = comp)
  mats <- equalize_lengths(mats)
  all_mat <- do.call(rbind, mats)

  k <- ncol(all_mat)
  cn <- colnames(all_mat)
  if (is.null(cn)) cn <- paste0(label, ".", seq_len(k))

  ## Common case: 3 deltas () (k == 3) → one row with Delta1(D1)..Delta3(D3)
  if (assume_3_deltas && k == 3) {
    meds <- apply(all_mat, 2, median)
    q025s <- apply(all_mat, 2, quantile, probs = 0.025)
    q975s <- apply(all_mat, 2, quantile, probs = 0.975)
    tab <- data.frame(
      Parameter = label,
      `D1_median` = meds[1], `D1_2.5%` = q025s[1], `D1_97.5%` = q975s[1],
      `D2_median` = meds[2], `D2_2.5%` = q025s[2], `D2_97.5%` = q975s[2],
      `D3_median` = meds[3], `D3_2.5%` = q025s[3], `D3_97.5%` = q975s[3],
      check.names = FALSE
    )
    tab[, -1] <- round(tab[, -1], 3)
    print(tab, row.names = FALSE)
    return(invisible(tab))
  }
}
```

```

## MBeta/MPsi-like case: multiple parameters × 3 deltas (k multiple of 3)
if (assume_3_deltas && k %% 3 == 0) {
  npar <- k / 3
  med_mat <- t(matrix(apply(all_mat, 2, median), 3, npar))
  q025_mat <- t(matrix(apply(all_mat, 2, quantile, probs=0.025), 3, npar))
  q975_mat <- t(matrix(apply(all_mat, 2, quantile, probs=0.975), 3, npar))

  if (is.null(param_names)) param_names <- paste0(label, "[", seq_len(npar))

  tab <- data.frame(
    Parameters = param_names,
    `D1_median` = med_mat[,1], `D1_2.5%` = q025_mat[,1], `D1_97.5%` = q975_mat[,1],
    `D2_median` = med_mat[,2], `D2_2.5%` = q025_mat[,2], `D2_97.5%` = q975_mat[,2],
    `D3_median` = med_mat[,3], `D3_2.5%` = q025_mat[,3], `D3_97.5%` = q975_mat[,3],
    check.names = FALSE
  )
  tab[, -1] <- round(tab[, -1], 3)
  print(tab, row.names = FALSE)
  return(invisible(tab))
}

## General fallback: long table, one row per column
med <- apply(all_mat, 2, median)
q025 <- apply(all_mat, 2, quantile, probs = 0.025)
q975 <- apply(all_mat, 2, quantile, probs = 0.975)
tab <- data.frame(Parameter = cn, Median = med, `2.5%` = q025, `97.5%` = q975,
  check.names = FALSE)
tab[, -1] <- round(tab[, -1], 3)
print(tab, row.names = FALSE)
invisible(tab)
}

## ===== Table 1 (pooled across all CODA chains) =====
## Assumes you already have `mods6 <- list(Mod30, Mod31, Mod32, Mod33, Mod34, Mod35)`
## and the helper functions `align_columns_list()` and `equalize_lengths()` in memory.

## 1) Collect MBeta matrices from all chains and ensure consistent shape/order
mats_beta <- lapply(mods6, function(m) as.matrix(m$MBeta)) # one MBeta per chain
mats_beta <- align_columns_list(mats_beta, what = "MBeta") # align columns across chains
mats_beta <- equalize_lengths(mats_beta) # truncate to common length

## 2) Pool samples by stacking rows (treating chains as independent draws)
MBeta_all <- do.call(rbind, mats_beta)

## 3) Compute posterior summaries using all pooled samples
med_mat <- t(matrix(apply(MBeta_all, 2, median), 3, 9))
q025_mat <- t(matrix(apply(MBeta_all, 2, quantile, probs=0.025), 3, 9))
q975_mat <- t(matrix(apply(MBeta_all, 2, quantile, probs=0.975), 3, 9))

## 4) Assemble the table
param_nms <- c("intercept", "Lon", "Lat", "East", "West",
  "North", "Demographic density", "Military police", "playground")

```

```

tab <- data.frame(
  Parameters = param_nms,
  `D1_median` = med_mat[,1], `D1_2.5%` = q025_mat[,1], `D1_97.5%` = q975_mat[,1],
  `D2_median` = med_mat[,2], `D2_2.5%` = q025_mat[,2], `D2_97.5%` = q975_mat[,2],
  `D3_median` = med_mat[,3], `D3_2.5%` = q025_mat[,3], `D3_97.5%` = q975_mat[,3],
  check.names = FALSE
)

## 5) Round and print
tab[, -1] <- round(tab[, -1], 3)
print(tab)

## ===== Table 2 (pooled across all CODA chains) =====
## Assumes you already have `mods6 <- list(Mod30, Mod31, Mod32, Mod33, Mod34, Mod35)`
## and the helper functions `align_columns_list()` and `equalize_lengths()` in memory.

## 1) Collect MPsi matrices from all chains and ensure consistent shape/order
mats_psi <- lapply(mods6, function(m) as.matrix(m$MPsi)) # one MPsi per chain
mats_psi <- align_columns_list(mats_psi, what = "MPsi") # align columns across chains
mats_psi <- equalize_lengths(mats_psi) # truncate to common length

## 2) Pool samples by stacking rows (treating chains as independent draws)
MPsi_all <- do.call(rbind, mats_psi)

## 3) Compute posterior summaries using all pooled samples
med_mat <- t(matrix(apply(MPsi_all, 2, median), 3, 9))
q025_mat <- t(matrix(apply(MPsi_all, 2, quantile, probs=0.025), 3, 9))
q975_mat <- t(matrix(apply(MPsi_all, 2, quantile, probs=0.975), 3, 9))

## 4) Assemble the table
param_nms <- c("intercept", "Lon", "Lat", "East", "West",
              "North", "Demographic density", "Military police", "playground")

tab <- data.frame(
  Parameters = param_nms,
  `D1_median` = med_mat[,1], `D1_2.5%` = q025_mat[,1], `D1_97.5%` = q975_mat[,1],
  `D2_median` = med_mat[,2], `D2_2.5%` = q025_mat[,2], `D2_97.5%` = q975_mat[,2],
  `D3_median` = med_mat[,3], `D3_2.5%` = q025_mat[,3], `D3_97.5%` = q975_mat[,3],
  check.names = FALSE
)

## 5) Round and print
tab[, -1] <- round(tab[, -1], 3)
print(tab)

```

Results produced by the code above:

Parameters	D1_median	D1_2.5	D1_97.5	D2_median	D2_2.5	D2_97.5	D3_median	D3_2.5	D3_97.5
intercept	-0.004	-31.221	31.118	0.027	-37.349	37.522	0.001	-38.385	38.443
Lon	-0.445	-3.492	2.539	-0.144	-3.199	3.114	-0.576	-3.790	2.444
Lat	0.009	-0.079	0.100	-0.002	-0.100	0.090	0.021	-0.071	0.119
East	-2.318	-3.912	-0.718	-2.529	-4.438	-0.895	-2.322	-3.987	-0.679
West	-2.579	-4.405	-0.931	-2.810	-5.031	-1.130	-2.574	-4.469	-0.905
North	-2.534	-5.205	0.003	-2.779	-5.890	-0.155	-2.473	-5.133	0.078
Demographic density	-0.242	-0.570	0.107	-0.240	-0.580	0.120	-0.237	-0.562	0.113
Military police	-0.177	-0.600	0.321	-0.178	-0.641	0.343	-0.144	-0.563	0.345
playground	0.406	-0.062	0.833	0.407	-0.083	0.852	0.396	-0.070	0.822

Parameters	D1_median	D1_2.5	D1_97.5	D2_median	D2_2.5	D2_97.5	D3_median	D3_2.5	D3_97.5
intercept	-0.004	-31.221	31.118	0.027	-37.349	37.522	0.001	-38.385	38.443
Lon	-0.445	-3.492	2.539	-0.144	-3.199	3.114	-0.576	-3.790	2.444
Lat	0.009	-0.079	0.100	-0.002	-0.100	0.090	0.021	-0.071	0.119
East	-2.318	-3.912	-0.718	-2.529	-4.438	-0.895	-2.322	-3.987	-0.679
West	-2.579	-4.405	-0.931	-2.810	-5.031	-1.130	-2.574	-4.469	-0.905
North	-2.534	-5.205	0.003	-2.779	-5.890	-0.155	-2.473	-5.133	0.078
Demographic density	-0.242	-0.570	0.107	-0.240	-0.580	0.120	-0.237	-0.562	0.113
Military police	-0.177	-0.600	0.321	-0.178	-0.641	0.343	-0.144	-0.563	0.345
playground	0.406	-0.062	0.833	0.407	-0.083	0.852	0.396	-0.070	0.822

Reproducibility of Figures

Below is the code to reproduce Figure 2. Note that with the command line

```
1 <-3
```

we select the neighborhood figure according to the order reported at the beginning of this document; for example, (1<-3) will produce the plot of the function m for the neighborhood *Candelária*.

```
## ===== Build the figure using SIX chains with per-chain thinning =====
## English (R-style) comments; your plotting block remains EXACTLY the same.
## Target: from each model, keep 3,000 draws spaced by 100 (stride = 100)
## -> 18,000 total after pooling.

## ----- Helpers (same behavior you used for CODA alignment) -----
align_columns_list <- function(mats, what = "MBeta") {
  stopifnot(length(mats) >= 2)
  cn1 <- colnames(mats[[1]])
  if (is.null(cn1)) {
    k <- ncol(mats[[1]])
    ok <- vapply(mats, function(M) ncol(M) == k, TRUE)
    if (!all(ok)) stop(sprintf("%s: different number of columns across chains.", what))
    return(mats)
  }
  for (i in seq_along(mats)) {
    cni <- colnames(mats[[i]])
    if (is.null(cni) || !setequal(cni, cn1))
      stop(sprintf("%s: different column sets across chains.", what))
    mats[[i]] <- mats[[i]][, cn1, drop = FALSE]
  }
}
```

```

  mats
}

equalize_lengths <- function(mats) {
  min_n <- min(vapply(mats, nrow, 1L))
  lapply(mats, function(M) M[(nrow(M) - min_n + 1):nrow(M), , drop = FALSE])
}

## ----- List of chains -----
mods6 <- list(Mod30, Mod31, Mod32, Mod33, Mod34, Mod35)

## ----- Extract MW/MM, align columns, equalize lengths -----
mats_MW <- lapply(mods6, function(m) as.matrix(m$MW)) # MW per chain
mats_MM <- lapply(mods6, function(m) as.matrix(m$MM)) # MM per chain

mats_MW <- align_columns_list(mats_MW, what = "MW")
mats_MM <- align_columns_list(mats_MM, what = "MM")

mats_MW <- equalize_lengths(mats_MW)
mats_MM <- equalize_lengths(mats_MM)

## ----- Per-chain thinning: 3,000 draws per chain with stride = 100 -----
n_keep <- 3000L # draws per chain to retain
stride <- 100L # step between retained draws

thin_chain <- function(M, n_keep, stride) {
  N <- nrow(M)
  start_idx <- max(1L, N - (n_keep - 1L) * stride) # back-fill from the end
  idx <- seq.int(from = start_idx, to = N, by = stride)
  ## Ensure exactly n_keep rows (trim or pad if off by one due to bounds)
  if (length(idx) > n_keep) idx <- tail(idx, n_keep)
  if (length(idx) < n_keep) idx <- c(rep(idx[1], n_keep - length(idx)), idx)
  # very unlikely
  M[idx, , drop = FALSE]
}

mats_MW <- lapply(mats_MW, thin_chain, n_keep = n_keep, stride = stride)
mats_MM <- lapply(mats_MM, thin_chain, n_keep = n_keep, stride = stride)

## ----- Pool thinned chains (final size: 6 * 3,000 = 18,000 draws) -----
MW <- do.call(rbind, mats_MW)
MM <- do.call(rbind, mats_MM)

## ----- Define time breaks and site/segment dimensions -----
ini<-c(0,breaks[1:(length(breaks)-1)])
fim<-breaks

n_seg <- length(fim)
stopifnot(ncol(MW) %% n_seg == 0L)
n_sites <- ncol(MW) %/% n_seg

## Dummy shape anchors so nrow(Wm)/ncol(Wm) are numeric (prevents extent errors)
Wm <- matrix(NA_real_, nrow = n_seg, ncol = n_sites)

```

```

Mm <- matrix(NA_real_, nrow = n_seg, ncol = n_sites)

## ===== YOUR ORIGINAL CODE (UNCHANGED) =====
mtnp <- function(dados) {
  a <- c(0, dados)
  m <- length(dados)
  k <- 1
  corr <- array(NA, dim = c(m, 2))
  for (i in 1:m) {
    corr[i, 1] <- (a[i] + a[i + 1]) / 2
    tt <- (a[i] + a[i + 1]) / 2
    rmt <- (1 / k) * (i - 1) + (tt - a[i]) / (k * (a[i + 1] - a[i]))
    corr[i, 2] <- rmt
  }
  corr
}
#
mf <- function(Wl, Ml, tau) {
  res <- exp(Wl) * tau^(exp(Ml))
  res
}
#
ml <- function(Wl, Ml, tini, tfim) {
  xx <- seq(tini, tfim, length = 100)
  res <- NULL

  for (i in 1:length(xx)) {
    aux <- exp(Wl) * xx[i]^(exp(Ml))
    res <- c(res, aux)
  }

  res1 <- list(res, xx)
  res1
}
#
mdil <- function(Wo, Mo, tauini, taufim) {
  L <- length(Wo)
  vecx <- NULL
  vecy <- NULL
  for (l in 1:L) {
    if (l == 1) {
      aux <- ml(Wo[l], Mo[l], tauini[l], taufim[l])
      vecx <- c(vecx, aux[[2]])
      vecy <- c(vecy, aux[[1]])
    } else {
      acum <- 0
      for (k in 1:(l - 1)) {
        aux1 <- mf(Wo[k], Mo[k], taufim[k]) - mf(Wo[k + 1], Mo[k + 1], taufim[k])
        acum <- acum + aux1
      }
      temp <- ml(Wo[l], Mo[l], tauini[l], taufim[l])
      aux2 <- acum + temp[[1]]
      vecx <- c(vecx, temp[[2]])
    }
  }
}

```

```

    vecy <- c(vecy, aux2)
  }
}

res <- list(vecx, vecy)
res
}
#
# Sites l=1,...,34
Bairro<-c("Lagoa Nova","Nova Descoberta","Candelária","Capim Macio","Pitimbu",
          "Neópolis","Ponta Negra","Santos Reis","Rocas","Ribeira","Praia do Meio",
          "Petrópolis","Areia Preta","Cidade Alta","Mãe Luiza","Alecrim",
          "Barro Vermelho","Tirol","Lagoa Seca","Cidade Esperança","Quintas",
          "Dix_Sept Rosado","Bom Pastor","Nossa Sra. de Nazaré","Felipe Camarão",
          "Cidade Nova","Guarapes","Planalto","Igapó","Potengi",
          "Nossa Sra. da Apresentação","Lagoa Azul","Pajuçara","Redinha")

l <-3

y1 <- NULL
for (h in 1:nrow(MW)) {
  Wme <- matrix(MW[h, ], nrow(Wm), ncol(Wm))
  Mme <- matrix(MM[h, ], nrow(Mm), ncol(Mm))
  lll <- mdil(Wme[, 1], Mme[, 1], ini, fim)
  y1 <- rbind(y1, t(as.matrix(lll[[2]])))
}

medy1 <- apply(y1, 2, median)
per25 <- apply(y1, 2, quantile, probs = 0.025)
per975 <- apply(y1, 2, quantile, probs = 0.975)
infy <- min(per25)
supy <- max(per975)
retemp <- lll[[1]]
infx <- min(retemp)
supx <- max(retemp)
xx <- c(retemp, rev(retemp))
yy <- c(per25, rev(per975))
plot(xx, yy, type = "n", xlab = " ", ylab = " ", xlim = c(infx, supx),
      ylim = c(infy, supy), cex.lab = 1.5)
polygon(xx, yy, col = "gray", border = NA)
par(new = T)
plot(mtnp(natal_theft_times[, 1]), type = "l", main = " ", xlim = c(infx, supx),
      ylim = c(infy, supy), xlab = " ", ylab = " ")
par(new = T)
plot(retemp, medy1,
      type = "l", xlim = c(infx, supx), ylim = c(infy, supy), xlab = " ",
      ylab = " ", main = Bairro[l], lwd = 2, cex.main = 2, col = 2
)

abline(v=fim[1],lty=2)
abline(v=fim[2],lty=2)

```

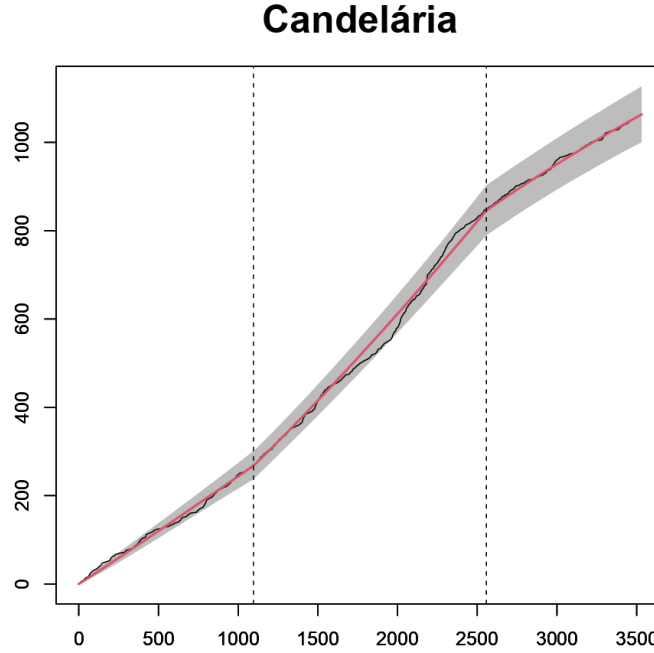


Figure 3 – Spatial pattern of α_j across neighborhoods and time intervals

What Figure 3 shows. Figure-3 was created in QGIS using the posterior summaries of α_j employed as thematic values for each neighborhood and time interval ($\Delta_1, \Delta_2, \Delta_3$). The table below reports, for each neighborhood and interval, the posterior mean and 95% credible interval (CrI) of $\alpha_j = \exp(M)$, where M denotes the corresponding log-intensity component from the fitted model.

How we produced the summaries for QGIS. Starting from the posterior draws matrix MM organized with 34×3 columns (34 neighborhoods, each with $\Delta_1, \Delta_2, \Delta_3$), we computed the columnwise summaries for $\alpha = \exp(M)$ and reshaped them into a wide table. The R code used is given below.

```
## ===== Table of alpha per neighborhood and interval (D1-D3) =====
## English (R-style) comments. Uses posterior draws in `MM` to summarize alpha = exp(M).
## Output: one wide table with mean and 95% CrI per D, rows in the canonical
## neighborhood order.

## ---- 0) Canonical neighborhood names in model order (34) ----
bairro_names <- c(
  # South Zone (7)
  "Lagoa Nova", "Nova Descoberta", "Candelária", "Capim Macio", "Pitumbu",
  "Neópolis", "Ponta Negra".
  # East Zone (12)
  "Santos Reis", "Rocas", "Ribeira", "Praia do Meio", "Petrópolis", "Areia Preta",
  "Cidade Alta", "Mãe Luiza", "Alecrim", "Barro Vermelho", "Tirol", "Lagoa Seca".
  # West Zone (9)
  "Cidade da Esperança", "Quintas", "Dix-Sept Rosado", "Bom Pastor",
  "Nossa Senhora de Nazaré", "Felipe Camarão", "Cidade Nova", "Guarapes", "Planalto".
  # North Zone (6)
```



```

    "Igapó", "Potengi", "Nossa Senhora da Apresentação", "Lagoa Azul", "Pajuçara", "Redinha"
  )

## ---- 1) Sanity check on MM layout: expect 34 * 3 columns (D1, D2, D3) ----
stopifnot(ncol(MM) == length(bairro_names) * 3)

## ---- 2) Column-wise summaries for alpha = exp(M) ----
Alpha_mean <- colMeans(exp(MM))
Alpha_q025 <- apply(exp(MM), 2, quantile, probs = 0.025)
Alpha_q975 <- apply(exp(MM), 2, quantile, probs = 0.975)
Alpha_median <- apply(exp(MM), 2, mean) # (optional placeholder;
medians can be used if desired)

## ---- 3) Reshape into 34 x 3 matrices (rows = neighborhoods, cols = D1..D3) ----
m_mean <- matrix(Alpha_mean, nrow = length(bairro_names), ncol = 3, byrow = FALSE,
                 dimnames = list(bairro_names, c("Delta 1", "Delta 2", "Delta 3")))
m_q025 <- matrix(Alpha_q025, nrow = length(bairro_names), ncol = 3, byrow = FALSE)
m_q975 <- matrix(Alpha_q975, nrow = length(bairro_names), ncol = 3, byrow = FALSE)
m_median <- matrix(Alpha_median, nrow = length(bairro_names), ncol = 3, byrow = FALSE)

## ---- 4) Assemble a single wide table (mean and 95% CrI per Delta (D))----
tab_alpha <- data.frame(
  Neighborhood = bairro_names,
  `D1_mean` = m_mean[,1], `D1_2.5%` = m_q025[,1], `D1_97.5%` = m_q975[,1],
  `D2_mean` = m_mean[,2], `D2_2.5%` = m_q025[,2], `D2_97.5%` = m_q975[,2],
  `D3_mean` = m_mean[,3], `D3_2.5%` = m_q025[,3], `D3_97.5%` = m_q975[,3],
  check.names = FALSE
)

## (Optional) also include medians instead of means:
# tab_alpha$`D1_median` <- m_median[,1]
# tab_alpha$`D2_median` <- m_median[,2]
# tab_alpha$`D3_median` <- m_median[,3]

## ---- 5) Round and print ----
tab_alpha[, -1] <- round(tab_alpha[, -1], 3)
print(tab_alpha, row.names = FALSE)

## ---- 6) Optional: pretty print or export ----
# if (requireNamespace("knitr", quietly = TRUE))
# knitr::kable(tab_alpha, align = "lrrrrrrrrrr")
# write.csv(tab_alpha, "alpha_by_neighborhood.csv", row.names = FALSE)

```

Posterior summaries used in QGIS.

Table~1 reports the α_j summaries used to build Figure~3.

Table 1: Posterior summaries of α_j by neighborhood and interval.

Neighborhood	$\Delta 1$ mean	$\Delta 1$ 2.5%	$\Delta 1$ 97.5%	$\Delta 2$ mean	$\Delta 2$ 2.5%	$\Delta 2$ 97.5%	$\Delta 3$ mean	$\Delta 3$ 2.5%	$\Delta 3$ 97.5%
Lagoa Nova	1.030	0.953	1.113	1.825	1.574	2.091	0.389	0.203	0.740
Nova Descoberta	1.285	1.126	1.444	0.443	0.204	0.930	1.023	0.877	1.180
Candelária	0.333	0.171	0.605	0.984	0.797	1.195	1.874	1.615	2.156
Capim Macio	1.026	0.892	1.162	1.487	1.163	1.794	0.424	0.226	0.758
Pitimbu	1.204	1.001	1.400	0.379	0.184	0.785	1.032	0.837	1.224
Neópolis	0.322	0.165	0.575	0.978	0.844	1.130	1.970	1.716	2.257
Ponta Negra	1.022	0.926	1.126	1.709	1.455	2.009	0.390	0.194	0.759
Santos Reis	1.247	1.058	1.436	0.371	0.178	0.789	1.103	0.911	1.291
Rocas	0.308	0.157	0.549	1.073	0.881	1.290	1.626	1.394	1.868
Ribeira	1.192	1.084	1.303	1.469	1.132	1.804	0.405	0.204	0.765
Praia do Meio	1.164	0.946	1.341	0.399	0.190	0.810	0.990	0.753	1.245
Petrópolis	0.314	0.144	0.566	1.035	0.934	1.142	1.741	1.407	2.131
Areia Preta	1.397	1.219	1.570	1.980	1.768	2.181	0.367	0.169	0.729
Cidade Alta	1.175	0.995	1.391	0.449	0.225	0.855	1.407	1.187	1.624
Mãe Luiza	0.325	0.150	0.591	1.193	1.064	1.337	1.700	1.446	1.912
Alecrim	1.190	1.067	1.323	1.854	1.621	2.080	0.443	0.208	0.809
Barro Vermelho	1.149	0.963	1.367	0.455	0.221	0.863	0.967	0.807	1.147
Tirol	0.325	0.144	0.567	1.170	1.037	1.313	2.192	1.909	2.564
Lagoa Seca	1.234	1.085	1.401	1.705	1.476	1.912	0.309	0.141	0.539
Cid. Esperança	1.120	0.894	1.414	0.439	0.210	0.886	1.122	0.981	1.267
Quintas	0.335	0.125	0.680	1.062	0.932	1.198	2.260	1.984	2.513
Dix-Sept Rosado	1.126	0.931	1.319	1.716	1.513	1.930	0.320	0.141	0.573
Bom Pastor	1.796	1.423	2.149	0.426	0.214	0.811	1.136	0.985	1.288
N.S. de Nazaré	0.381	0.163	0.836	1.167	0.990	1.353	2.367	2.174	2.592
Felipe Camarão	0.913	0.726	1.124	1.931	1.719	2.173	0.326	0.143	0.577
Cidade Nova	1.604	1.290	1.945	0.469	0.230	0.834	1.020	0.843	1.207
Guarapes	0.357	0.172	0.730	0.871	0.736	1.024	2.273	2.044	2.555
Planalto	0.978	0.816	1.156	2.034	1.701	2.381	0.308	0.132	0.567
Igapó	1.640	1.376	1.968	0.412	0.216	0.767	1.096	0.925	1.275
Potengi	0.371	0.168	0.790	1.078	0.934	1.247	2.134	1.884	2.377
N.S. Apresentação	0.949	0.782	1.141	1.952	1.683	2.197	0.309	0.141	0.560
Lagoa Azul	1.582	1.274	1.904	0.479	0.248	0.833	1.054	0.837	1.295
Pajuçara	0.378	0.178	0.817	0.897	0.743	1.065	1.856	1.481	2.183
Redinha	1.142	0.992	1.304	2.048	1.712	2.365	0.316	0.138	0.627

Note: Values are posterior means and 95% credible intervals (2.5%–97.5%) for $\alpha_j = \exp(M)$ by neighborhood and interval Δ_1 – Δ_3 .

Figures 4 and 5

The R code to reproduce Figures 4 and 5 is available in the code.txt file. To ensure full reproducibility, run the scripts in the order they appear in that file, as some objects and parameters used in the figures are created in earlier steps prior to the figure-specific code.

References

- Castro Morales, Fidel Ernesto. 2025a. “STPoissonSS: Bayesian Estimation for Spatiotemporal Nonhomogeneous Poisson Model with Weibull Intensity and State-Space Prior.” <https://github.com/Projeto-CNPq-Clima/STPoissonSS>.
- . 2025b. “MCMC Estimation Outputs for the Article Assessing the Impact of Public Policies on Vehicle Theft Incidence in Natal: A Spatial-Temporal Model Using Non-Homogeneous Poisson Processes.” Zenodo. <https://doi.org/10.5281/zenodo.17011948>.